

Agent Development Life Cycle (ADLC)

By

Anand Kulkarni

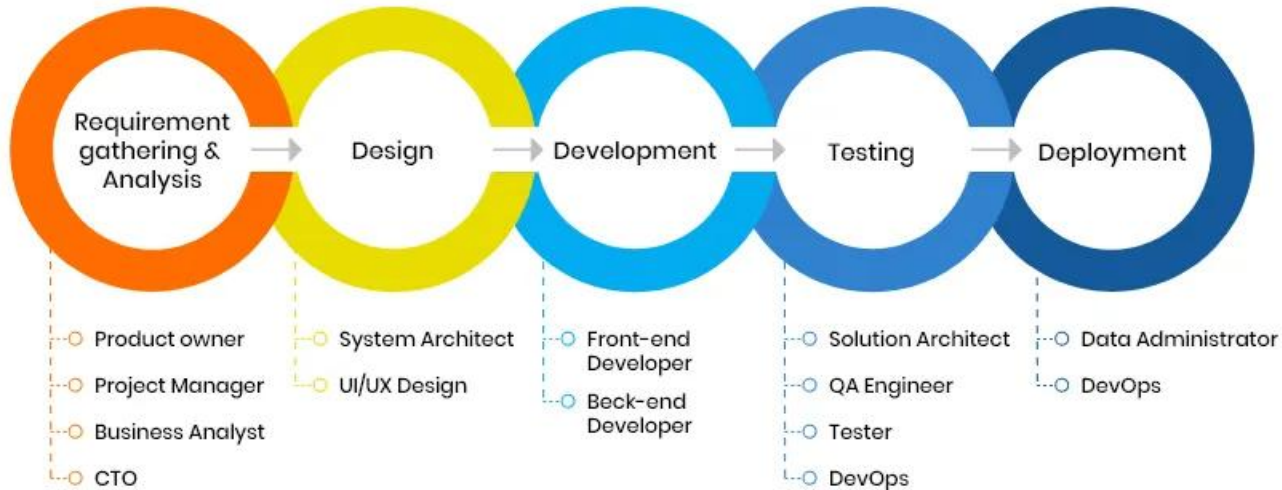
anand.kulkarni1@zensar.com

Table of Content

Module	Topic
Module 1:	Introduction to SDLC
Module 2:	Introduction to ADLC
Module 3:	ADLC stages
Module 4:	SDLC vs ADLC
Module 5:	Example

Introduction to SDLC

5 Phases of the Software Development Life Cycle



SDLC phases

1. Planning and Requirement Analysis

This initial stage involves gathering requirements from stakeholders, customers & market surveys. The goal is to create a basic project plan that outlines the project's scope, objectives, and feasibility.

2. Defining Requirements

In this stage, detailed requirements for the software are specified & documented in a Software Requirement Specification (SRS). SRS serves as a reference for the entire development process.

3. Designing Architecture

Based on the SRS, multiple design options are created and evaluated. The best design is chosen and documented in the Design Document Specification (DDS). This stage ensures that the software's architecture is well-defined.

SDLC phases

4. Developing the Product

This is the coding phase where developers write the actual code based on the design specifications. Various programming languages and tools are used to create the software.

5. Testing and Integration

After development, the software undergoes rigorous testing to identify and fix any bugs. This stage ensures that the software meets the quality standards and functions as expected.

6. Deployment and Maintenance

The final product is deployed in phases and tested in a real environment. Feedback is collected, and necessary improvements are made. Maintenance involves fixing bugs, updating the software, and ensuring its smooth operation.

Agent Development Life Cycle (ADLC)

- The Agent Development Life Cycle (ADLC) is a structured process for designing, developing, deploying, and continuously improving an AI agent.
- Unlike a traditional software application, an AI agent may need to reason about a task, use tools, retrieve information, make decisions, interact with other systems, and adapt its behavior based on feedback. Therefore, agent development requires additional stages around evaluation, safety, prompting, tool use, and continuous monitoring.
- A typical ADLC can be represented as:

Problem Definition → Agent Design → Data & Knowledge → Build → Evaluation → Deployment → Monitoring → Continuous Improvement

ADLC Stages

1. Problem Definition: The first step is to identify-

- What problem should the agent solve?
- Who will use it?
- What decisions or actions can it take?
- What systems does it need to interact with?
- What are the boundaries of the agent?
- What does success look like?

For example, instead of saying "Build a customer-support agent," we might define the problem as:

"Build an agent that can answer customer questions, retrieve order information, initiate eligible refunds, and escalate complex issues to a human agent." This creates clear boundaries and measurable objectives.

ADLC Stages

2. Agent Design

In this stage, we define how the agent will operate. Key design decisions include:

Agent type: conversational, autonomous, workflow-based, multi-agent, etc.

Reasoning approach: how the agent determines what to do next.

Tools: APIs, databases, search, calculators, CRM systems, etc.

Memory: what information should be retained during or across conversations.

Knowledge: what documents or enterprise data the agent can access.

Guardrails: what the agent is allowed and not allowed to do.

Human-in-the-loop: when a human must approve or take over.

ADLC Stages

3. Knowledge and Data Preparation

Agents often depend on external knowledge. This could include:

Product documentation, FAQs, Company policies, Databases, Customer records, Internal knowledge bases, APIs

For example, a support agent shouldn't simply guess a refund policy. It should retrieve the company's current refund policy from an approved knowledge source.

This stage can involve RAG (Retrieval-Augmented Generation), data cleaning, document chunking, embeddings, metadata, access controls, and knowledge-source validation.

ADLC Stages

4. Build and Integration

Now the agent is implemented. Typical components of our agent include:

LLM/model, System instructions/prompts, Tool/function calling, RAG pipeline, Memory, API integrations, Workflow/orchestration, Authentication and authorization, Logging and tracing, Guardrails

The important difference from ordinary application development is that part of the system's behavior is probabilistic rather than completely deterministic.

ADLC Stages

5. Evaluation

Evaluation is one of the most important parts of ADLC. Traditional software testing often asks:

"Does the application produce the expected output for this input?"

For agents, we also need to ask:

"Did the agent choose the right approach, use the right tools, follow policies, and produce an acceptable result?"

Evaluation can cover:

Accuracy, Task completion, Tool selection, Tool-call correctness, Groundedness, Hallucination rate, Safety, Latency, Cost, Robustness, Prompt-injection resistance, Human satisfaction.

ADLC Stages

5. Evaluation

For example: User: "Cancel my order and give me a refund."

The evaluation isn't simply whether the agent generated a good sentence. We need to check whether it:

- Identified the customer's order.
- Checked whether the order was eligible for cancellation.
- Retrieved the refund policy.
- Used the correct cancellation API.
- Confirmed the action.
- Didn't perform an unauthorized refund.

ADLC Stages

6. Deployment

After evaluation, the agent is released into production. Agent Deployment may involve:

Model/API configuration, Infrastructure, Authentication, Rate limits, Monitoring, Logging, Version management, Rollback mechanisms, Security controls

Often, organizations initially deploy agents to a limited group of users before expanding their usage.

ADLC Stages

7. Monitoring and Observability

This is especially important for agents because their behavior can change when:

The model changes

Prompts change

Knowledge sources change

Tools/API behavior changes

User behavior changes

New edge cases appear

ADLC Stages

7. Monitoring and Observability

Monitoring should therefore track things such as:

Success/failure rate, Tool-call failures, Hallucinations, Escalations, User feedback, Latency, Token usage, Cost, Safety violations

Agent traces can also be reviewed to understand what the agent attempted to do and where it failed.

ADLC Stages

8. Continuous Improvement

ADLC is generally a continuous loop, rather than a one-time process. Production feedback leads to:

Monitor → Identify failure → Analyze root cause → Modify agent → Evaluate → Redeploy

For example, if an agent frequently gives incorrect answers about a newly introduced product, the solution might be:

Update the knowledge base, Improve retrieval, Change instructions, Add evaluation cases, Add a guardrail, Change the tool/API logic

The agent is then reevaluated before the change reaches production.

SDLC vs ADLC

Area	SDLC	ADLC
Primary focus	Software/application	AI agents
Requirements	Functional & non-functional requirements	Goals, autonomy, boundaries, policies
Core logic	Mostly deterministic code	Code + models + prompts + reasoning
Data	Usually application data	Data + knowledge + context
Testing	Unit, integration, system testing	Evaluation + testing + behavioral assessment
Output	Usually predictable	Can be probabilistic
Tool usage	Explicitly programmed workflows	Agent may dynamically select tools
Knowledge	Often encoded in application/database	May require RAG/knowledge retrieval

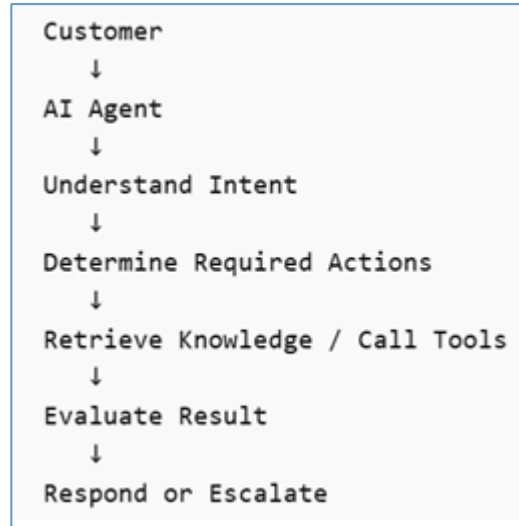
SDLC vs ADLC

Area	SDLC	ADLC
Memory	Database/session state	Conversation/agent memory may be required
Safety	Security & access controls	Security + AI safety + hallucination/guardrails
Monitoring	Errors, performance, availability	Errors + model behavior + quality + cost + safety
Improvement	New software releases	Continuous evaluation and model/agent improvement
Human involvement	Usually defined workflows	May include human-in-the-loop decisions

Example

Imagine a company wants to build an AI Customer Support Agent.

Traditional SDLC approach



Example

ADLC approach

Customer



AI Agent



Understand Intent



Determine Required Actions|



Retrieve Knowledge / Call Tools



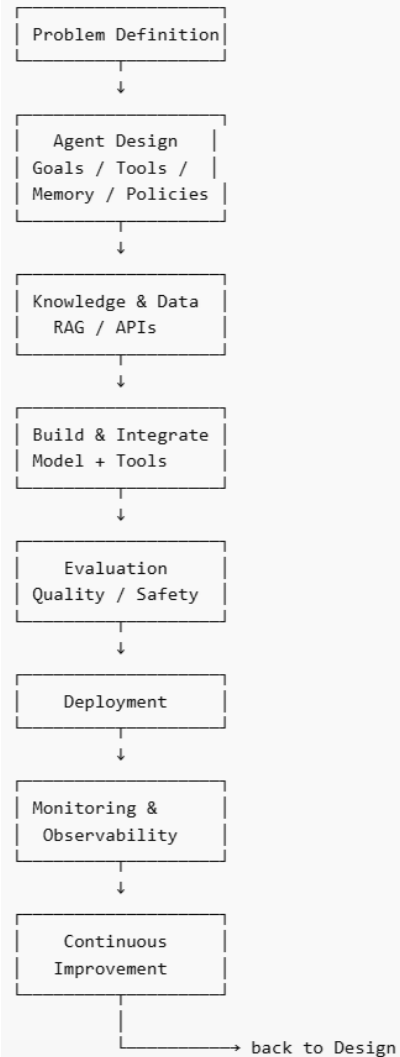
Evaluate Result



Respond or Escalate

Example

ADLC diagram



Thank you!!